SPÉCIFICATION FONCTIONNELLE

Assist

L'assistant développeur intelligent propulsé par Claude AI

Full-Stack : API REST + Frontend (React / Angular)

Version 1.1 — Mars 2026 — Statut : **Draft**

Champ	Valeur
Projet	Assist — Assistant développeur IA (Full-Stack)
Backend	Python (FastAPI) | JavaScript (Express) | Java (Spring Boot) | PHP (Laravel)
Frontend	React (+ Tailwind CSS) ou Angular (+ Angular Material) — au choix
Type de document	Spécification fonctionnelle
Version	1.1 (ajout du frontend)
Dernière mise à jour	Mars 2026

1. Vision et contexte

1.1 Objectif du produit

Assist est un assistant développeur intelligent full-stack propulsé par Claude AI. Il offre une interface conversationnelle (chat) permettant la recherche de documentation, la génération de code, la revue de PR, l'analyse de bugs et l'orchestration de tâches complexes via des agents IA spécialisés.

1.2 Problème adressé

- Les développeurs passent 30 à 50% de leur temps à chercher de l'information dans la documentation et le code source.
- Le context switching entre outils dégrade la productivité.
- La revue de code manuelle est chronophage et sujette aux oublis.
- La documentation technique est souvent obsolète ou incomplète.
- Les outils IA existants manquent d'une interface unifiée et contextualisée.

1.3 Proposition de valeur

Assist centralise ces tâches dans une application web full-stack avec un chat intelligent qui comprend le contexte du projet, exécute des actions concrètes via des outils, et coordonne des agents spécialisés.

1.4 Périmètre

In Scope (v1.1)

API REST conversationnelle avec streaming SSE
Frontend SPA (React ou Angular) : interface de chat temps réel
Recherche sémantique dans la documentation projet
Tool calling : recherche, exécution de tests, analyse de code
Analyse d'images (captures d'écran d'erreurs) via Vision API
Serveur MCP exposant les outils Assist
Orchestration multi-agents (analyse, tests, documentation)

Out of Scope (v1.1)

Authentification utilisateur et gestion de rôles
Persistance en base de données (in-memory uniquement)

Déploiement cloud / containerisation
Application mobile native
Support multi-utilisateurs temps réel (WebSocket)

2. Personas et acteurs

Persona	Description	Besoins clés
Développeur junior	Peu d'expérience, besoin d'aide pour comprendre le code	Recherche doc, explication de code, guidance
Développeur confirmé	Autonome, cherche à gagner du temps	Génération de code, revue auto, refactoring
Tech Lead	Responsable qualité et architecture	Revue de code, documentation auto, métriques
DevOps / SRE	Gère pipelines et infrastructure	Analyse de logs, scripts, debugging

2.1 Acteurs système

- Utilisateur : interagit avec Assist via le frontend React/Angular
- Frontend SPA : application web single-page (React ou Angular)
- Backend API : serveur REST qui orchestre les appels à Claude
- Claude API : fournit les capacités d'IA (génération, raisonnement, vision)
- Système de fichiers : source de la documentation et du code projet
- Serveur MCP : expose les outils Assist aux clients MCP externes
- Agents spécialisés : CodeAnalyzer, TestRunner, DocWriter

3. Architecture frontend

3.1 Choix technologique

Le participant choisit librement entre React et Angular. Les deux options sont équivalentes en termes de fonctionnalités. Le frontend est une Single Page Application (SPA) de type chat.

	Option React	Option Angular
Framework	React 18+ (Vite)	Angular 17+ (CLI)
UI Library	Tailwind CSS	Angular Material
State	useState / useReducer	Services + RxJS
HTTP Client	fetch / axios	HttpClient
Streaming SSE	EventSource API	EventSource API
Build	npm run build (Vite)	ng build
Dev Server	npm run dev (port 5173)	ng serve (port 4200)

3.2 Structure de l'interface

L'interface est une page unique de chat, inspirée de l'esthétique de Claude.ai, avec les zones suivantes :

```
┌──────────────────────────────────────────────────────────┐
│ 🤖 Assist                          [Modèle ▾] [⚙]  │  ← Header
├──────────────────────────────────────────────────────────┤
│                                                          │
│                                                          │
│   ┌──────────────────────────────────────┐               │
│   │ 👤 Comment configurer le streaming ?  │               │  ← Message user
│   └──────────────────────────────────────┘               │
│                                                          │
│                                                          │
│   ┌──────────────────────────────────────┐               │
│   │ 🤖 Pour configurer le streaming SSE...  │               │  ← Message assistant
│   │    📎 Source : docs/streaming.md          │               │   ← Source (tool)
│   │    ```code block```                       │               │   ← Code formaté
│   └──────────────────────────────────────┘               │
│                                                          │
│                                                          │
│   ┌──────────────────┐                                   │
│   │ 🔧 search_docs ✓  │                                   │  ← Tool indicator
│   │ 🔧 run_tests ⏳   │                                   │  ← Tool en cours
│   └──────────────────┘                                   │
│                                                          │
```

```
└──────────────────────────────────────────────────┘
  [📎] Tapez votre message...            [Envoyer ➢]   ← Input + upload
```

3.3 Composants frontend

Composant	Responsabilité	Module lié
ChatContainer	Layout principal, gestion du scroll auto	M2
MessageBubble	Affichage d'un message (user ou assistant)	M2
MessageInput	Zone de saisie + bouton envoi + upload image	M2
StreamingText	Rendu progressif du texte en streaming	M2
MarkdownRenderer	Rendu Markdown + coloration syntaxique code	M2
ToolIndicator	Badge montrant l'état d'exécution d'un tool	M4
ToolResultCard	Carte affichant le résultat d'un tool	M4
ImageUpload	Upload et preview d'une capture d'écran	M4
AgentReport	Carte affichant le rapport d'un agent	M6
OrchestratorProgress	Timeline de progression des agents	M6
ModelSelector	Dropdown pour changer de modèle Claude	M1
ConfigPanel	Panneau latéral de configuration (optionnel)	M1

4. Spécification détaillée par module

4.1 Module M1 — Core API + Configuration frontend

Infrastructure de base côté backend et configuration accessible depuis le frontend.

Endpoints

```
GET  /health         → { status, version, uptime, model }
GET  /config         → { model, max_tokens, temperature }
POST /config         → Mettre à jour la configuration runtime
```

User Stories

M1-US01 vérifier que le serveur Assist est opérationnel via GET /health

En tant que **développeur**, je veux **vérifier que le serveur Assist est opérationnel via GET /health**
afin de *m'assurer que le service est disponible*

Critères d'acceptation :

- ✓ GET /health retourne 200 avec { status: 'ok', version: '1.0.0', uptime: number }
- ✓ Si la clé API est absente → { status: 'degraded', reason: 'missing_api_key' }

M1-US02 configurer le modèle Claude depuis le frontend

En tant que **développeur**, je veux **configurer le modèle Claude depuis le frontend**
afin de *adapter le compromis qualité/coût/vitesse selon mon besoin*

Critères d'acceptation :

- ✓ GET /config retourne la configuration incluant model, max_tokens, temperature
- ✓ POST /config accepte un body JSON partiel
- ✓ Les valeurs invalides retournent 400 avec un message explicite
- ✓ Modèle par défaut : claude-sonnet-4-5-20250514

M1-US03 changer de modèle Claude via un dropdown dans le header du chat

En tant que **développeur**, je veux **changer de modèle Claude via un dropdown dans le header du chat**
afin de *switcher rapidement entre Opus, Sonnet et Haiku sans quitter l'interface*

FRONTEND *React ou Angular — au choix du participant*

Critères d'acceptation :

- ✓ Un composant ModelSelector affiche le modèle courant dans le header
- ✓ Le dropdown liste : Opus 4.6, Sonnet 4.5, Haiku 4.5
- ✓ Le changement appelle POST /config et met à jour l'affichage
- ✓ Un indicateur visuel confirme le changement (toast ou badge)

M1-US04 recevoir des messages d'erreur structurés et affichés dans le cha...

En tant que **développeur**, je veux **recevoir des messages d'erreur structurés et affichés dans le chat**

afin de *comprendre et corriger mes problèmes*

Critères d'acceptation :

- ✓ Toutes les erreurs API retournent { error: string, message: string, details?: any }
- ✓ Le frontend affiche les erreurs dans une bulle distincte (rouge/warning)
- ✓ Les erreurs réseau affichent un message 'Connexion perdue' avec un bouton Réessayer

4.2 Module M2 — Conversation + Interface Chat

Interaction conversationnelle backend + interface chat frontend avec streaming temps réel.

Endpoints

```
POST /chat           → Réponse complète (bloquant)
POST /chat/stream    → Réponse en streaming SSE
GET  /chat/history   → Historique de la session
DELETE /chat/history  → Réinitialiser l'historique
```

User Stories — Backend

M2-US01 poser une question à Assist et recevoir une réponse complète

En tant que **développeur**, je veux **poser une question à Assist et recevoir une réponse complète**
afin de *obtenir de l'aide sur mon code ou ma documentation*

Critères d'acceptation :

- ✓ POST /chat accepte { message: string, session_id?: string }
- ✓ Retourne { response: string, session_id: string, usage: { input_tokens, output_tokens } }
- ✓ Le system prompt positionne Assist comme un assistant développeur
- ✓ Timeout de 60 secondes avec erreur explicite

M2-US02 recevoir la réponse en streaming token par token

En tant que **développeur**, je veux **recevoir la réponse en streaming token par token**
afin de *voir la réponse se construire en temps réel*

Critères d'acceptation :

- ✓ POST /chat/stream retourne un flux SSE (Content-Type: text/event-stream)
- ✓ Chaque event : { type: 'delta'|'done'|'error', content?: string }
- ✓ Le dernier event est 'done' avec les métriques d'usage
- ✓ Le client peut fermer la connexion (annulation)

M2-US03 avoir un historique persistant au sein d'une session

En tant que **développeur**, je veux **avoir un historique persistant au sein d'une session**
afin de *poser des questions de suivi sans répéter le contexte*

Critères d'acceptation :

- ✓ Historique stocké en mémoire, indexé par session_id
- ✓ Limité à 20 messages (FIFO)
- ✓ GET /chat/history retourne la liste avec role et content
- ✓ Sessions inactives 1h → nettoyage automatique

User Stories — Frontend Chat

M2-US04 voir les messages dans une interface de chat avec bulles différen...

En tant que **développeur**, je veux **voir les messages dans une interface de chat avec bulles différenciées**

afin de *distinguer visuellement mes messages des réponses de Assist*

FRONTEND *React ou Angular — au choix du participant*

Critères d'acceptation :

- ✓ Les messages utilisateur sont alignés à droite avec un fond coloré (ex: bleu)
- ✓ Les messages Assist sont alignés à gauche avec un fond neutre
- ✓ Chaque bulle affiche l'horodatage (HH:mm)
- ✓ Le chat scroll automatiquement vers le bas à chaque nouveau message
- ✓ Le scroll auto se désactive si l'utilisateur remonte manuellement

M2-US05 voir la réponse de Assist s'afficher progressivement en streaming

En tant que **développeur**, je veux **voir la réponse de Assist s'afficher progressivement en streaming**

afin de *percevoir un temps de réponse court et suivre le raisonnement*

FRONTEND *React ou Angular — au choix du participant*

Critères d'acceptation :

- ✓ Le frontend se connecte à POST /chat/stream via l'API EventSource ou fetch+ReadableStream
- ✓ Les tokens sont affichés au fur et à mesure dans la bulle assistant
- ✓ Un curseur clignotant (▌) est affiché pendant le streaming
- ✓ Le bouton Envoyer se transforme en bouton Stop pendant le streaming
- ✓ Cliquer sur Stop ferme la connexion SSE et affiche la réponse partielle

M2-US06 voir le contenu Markdown et le code formatés dans les réponses

En tant que **développeur**, je veux **voir le contenu Markdown et le code formatés dans les réponses**

afin de *lire du code avec coloration syntaxique directement dans le chat*

FRONTEND *React ou Angular — au choix du participant*

Critères d'acceptation :

- ✓ Le Markdown est rendu : gras, italique, listes, liens, headers
- ✓ Les blocs de code sont affichés avec coloration syntaxique (highlight.js ou Prism)
- ✓ Un bouton 'Copier' est présent sur chaque bloc de code
- ✓ Le langage du bloc est détecté et affiché (ex: python, javascript)

M2-US07 taper mon message et l'envoyer via le champ de saisie

En tant que **développeur**, je veux **taper mon message et l'envoyer via le champ de saisie**

afin de *interagir naturellement avec Assist*

FRONTEND *React ou Angular — au choix du participant*

Critères d'acceptation :

- ✓ Le champ de saisie est en bas de page, pleine largeur
- ✓ Envoi via bouton Envoyer ou touche Entrée (Shift+Entrée pour saut de ligne)
- ✓ Le champ se redimensionne automatiquement (auto-grow, max 5 lignes)

- ✓ Le champ est désactivé pendant le streaming (indicateur de chargement)
- ✓ Le champ est vidé après envoi

M2-US08 démarrer une nouvelle conversation

En tant que **développeur**, je veux **démarrer une nouvelle conversation** afin de *repartir d'un contexte vierge sans recharger la page*

FRONTEND *React ou Angular — au choix du participant*

Critères d'acceptation :

- ✓ Un bouton 'Nouvelle conversation' est visible dans le header
- ✓ Cliquer appelle DELETE /chat/history et vide l'affichage
- ✓ Un message d'accueil de Assist est affiché dans la conversation vide
- ✓ Le session_id est régénéré côté frontend

4.3 Module M3 — Search

Recherche dans la documentation projet (fichiers Markdown, texte) via requêtes en langage naturel.

Endpoint

```
POST /search          → Recherche par mots-clés avec scoring
```

User Stories

M3-US01 rechercher dans la doc avec une requête en langage naturel

En tant que **développeur**, je veux **rechercher dans la doc avec une requête en langage naturel**
afin de *trouver rapidement l'information sans parcourir tous les fichiers*

Critères d'acceptation :

- ✓ POST /search accepte { query: string, max_results?: number, include_content?: boolean }
- ✓ query requis, 1-500 caractères ; max_results optionnel, 1-50, défaut 5
- ✓ Retourne { results: [{ title, path, score, snippet, content? }], total, query_time_ms }
- ✓ Résultats triés par score décroissant ; snippet max 200 caractères

M3-US02 que la recherche couvre tous les .md et .txt de docs/

En tant que **développeur**, je veux **que la recherche couvre tous les .md et .txt de docs/**
afin de *qu'aucune documentation ne soit ignorée*

Critères d'acceptation :

- ✓ Indexation récursive de docs/ (.md et .txt)
- ✓ Fichiers > 1 Mo ignorés (log warning)
- ✓ Score calculé par fréquence de termes normalisée
- ✓ Recherche insensible à la casse et aux accents

M3-US03 recevoir une erreur claire si ma requête est invalide

En tant que **développeur**, je veux **recevoir une erreur claire si ma requête est invalide**
afin de *corriger sans deviner*

Critères d'acceptation :

- ✓ Query manquant → 400 { error: 'INVALID_QUERY' }
- ✓ Query > 500 chars → 400 { error: 'INVALID_QUERY' }
- ✓ max_results hors limites → 400 { error: 'INVALID_PARAMETER' }
- ✓ Dossier docs/ inexistant → 200 { results: [], total: 0 }

4.4 Module M4 — Tool Calling + Affichage frontend

Capacités d'action via function calling + affichage des outils et de la Vision API dans le chat.

Tools disponibles

Tool	Description	Input	Output
search_docs	Recherche documentation	{ query, max_results? }	{ results: SearchResult[] }
run_tests	Lance les tests du projet	{ scope?, file? }	{ passed, failed, errors, output }
analyze_code	Analyse statique d'un fichier	{ file_path }	{ issues: Issue[], metrics }

User Stories — Backend

M4-US01 que Assist recherche automatiquement dans la doc quand c'est pert...

En tant que **développeur**, je veux **que Assist recherche automatiquement dans la doc quand c'est pertinent**
afin de *obtenir des réponses contextualisées sans faire la recherche moi-même*

Critères d'acceptation :

- ✓ Claude appelle search_docs quand le message concerne la documentation
- ✓ Le résultat est injecté dans le contexte de conversation
- ✓ Claude cite les sources trouvées dans sa réponse
- ✓ Si aucun résultat → Claude l'indique et propose des alternatives

M4-US02 demander à Assist de lancer mes tests et interpréter les résultat...

En tant que **développeur**, je veux **demander à Assist de lancer mes tests et interpréter les résultats**
afin de *comprendre rapidement ce qui échoue*

Critères d'acceptation :

- ✓ run_tests exécute la commande de test configurée (npm test, pytest, mvn test, phpunit)
- ✓ Sortie capturée (stdout+stderr) et transmise à Claude
- ✓ Claude analyse les échecs et propose des corrections
- ✓ Timeout 120 secondes

M4-US03 demander à Assist d'analyser un fichier source

En tant que **développeur**, je veux **demander à Assist d'analyser un fichier source**
afin de *identifier problèmes, bugs et améliorations*

Critères d'acceptation :

- ✓ analyze_code lit le fichier et l'envoie à Claude

- ✓ Retourne issues[] avec severity, line, message + métriques
- ✓ Fichiers > 500 lignes → warning
- ✓ Path traversal (../) bloqué

M4-US04 que Assist chaîne plusieurs tools automatiquement

En tant que **développeur**, je veux **que Assist chaîne plusieurs tools automatiquement**
afin de *obtenir une réponse complète en une seule demande*

Critères d'acceptation :

- ✓ Claude peut appeler plusieurs tools séquentiellement
- ✓ Boucle tool_use → tool_result jusqu'à stop_reason='end_turn'
- ✓ Maximum 5 appels par requête (anti-boucle infinie)

User Stories — Frontend Tools

M4-US05 voir quels tools sont appelés par Assist pendant qu'il réfléchit

En tant que **développeur**, je veux **voir quels tools sont appelés par Assist pendant qu'il réfléchit**
afin de *comprendre ce que l'IA fait en coulisses*

FRONTEND *React ou Angular — au choix du participant*

Critères d'acceptation :

- ✓ Quand un tool_use est reçu, un badge ToolIndicator apparaît sous la bulle
- ✓ Le badge affiche : icône + nom du tool + état (⏳ en cours / ✓ terminé / ❌ erreur)
- ✓ Les badges restent visibles après la réponse finale
- ✓ L'animation de chargement (spinner) est affichée pendant l'exécution

M4-US06 voir les résultats des tools dans des cartes structurées

En tant que **développeur**, je veux **voir les résultats des tools dans des cartes structurées**
afin de *exploiter les résultats (recherche, tests, analyse) visuellement*

FRONTEND *React ou Angular — au choix du participant*

Critères d'acceptation :

- ✓ Les résultats de search_docs s'affichent dans une carte avec titre, path et snippet
- ✓ Les résultats de run_tests affichent un résumé coloré : X passed (vert), Y failed (rouge)
- ✓ Les résultats de analyze_code affichent la liste des issues avec icônes par severity
- ✓ Les cartes sont collapsibles (clic pour déplier les détails)

M4-US07 uploader une capture d'écran d'erreur dans le chat

En tant que **développeur**, je veux **uploader une capture d'écran d'erreur dans le chat**
afin de *obtenir un diagnostic visuel sans recopier le message*

FRONTEND *React ou Angular — au choix du participant*

Critères d'acceptation :

- ✓ Un bouton 📎 à côté du champ de saisie ouvre un sélecteur de fichier (images uniquement)
- ✓ L'image uploadée s'affiche en preview dans le champ de saisie avant envoi

- ✓ Formats acceptés : PNG, JPEG, GIF, WebP — max 5 Mo
- ✓ L'image est convertie en base64 et envoyée avec le message
- ✓ Le drag & drop d'image dans la zone de chat est supporté
- ✓ L'image s'affiche dans la bulle utilisateur après envoi

4.5 Module M5 — Serveur MCP

Expose les capacités de Assist via le Model Context Protocol pour l'intégration avec Claude Desktop.

Composants MCP

Type	Nom	Description
Tool	search_project_docs	Recherche dans la documentation
Tool	get_file_content	Lit un fichier du projet
Tool	list_project_files	Liste les fichiers d'un répertoire
Resource	project-readme	Expose le README.md comme contexte

User Stories

M5-US01 utiliser les outils Assist depuis Claude Desktop

En tant que **développeur**, je veux **utiliser les outils Assist depuis Claude Desktop** afin de *avoir l'aide de Assist sans quitter mon interface habituelle*

Critères d'acceptation :

- ✓ Serveur MCP démarre via stdio
- ✓ 3 tools découvrables via tools/list
- ✓ Resource project-readme via resources/list
- ✓ Configuration Claude Desktop documentée

M5-US02 rechercher dans la doc depuis Claude Desktop

En tant que **développeur**, je veux **rechercher dans la doc depuis Claude Desktop** afin de *trouver de l'info sans ouvrir le navigateur*

Critères d'acceptation :

- ✓ search_project_docs accepte { query } et retourne les résultats
- ✓ Comportement identique à POST /search
- ✓ Erreurs retournées comme contenu texte

M5-US03 lire un fichier projet depuis Claude Desktop

En tant que **développeur**, je veux **lire un fichier projet depuis Claude Desktop** afin de *consulter du code sans changer de fenêtre*

Critères d'acceptation :

- ✓ get_file_content accepte { path } relatif à la racine
- ✓ Fichiers binaires → erreur
- ✓ Fichiers > 100 Ko → tronqués
- ✓ Path traversal bloqué

M5-US04 avoir le README.md automatiquement disponible dans Claude Desktop

En tant que **développeur**, je veux **avoir le README.md automatiquement disponible dans Claude Desktop**

afin de *que Claude comprenne mon projet sans explication*

Critères d'acceptation :

- ✓ Resource expose le contenu du README.md
- ✓ README inexistant → message par défaut
- ✓ Contenu rechargé à chaque accès

4.6 Module M6 — Orchestration multi-agents + Dashboard agents

Coordination de plusieurs agents IA spécialisés avec visualisation de la progression dans le frontend.

Agents

Agent	Rôle	Tools
CodeAnalyzer	Analyse qualité du code, identifie bugs et améliorations	analyze_code, get_file_content
TestRunner	Exécute et interprète les résultats de tests	run_tests, analyze_code
DocWriter	Génère de la documentation technique à jour	get_file_content, search_docs

Endpoint

```
POST /orchestrate    → Exécuter une tâche multi-agents

Request:  { task: string, target?: string, agents?: string[] }
Response: { report: AgentReport[], summary: string, duration_ms: number }
```

User Stories — Backend

M6-US01 demander une analyse complète d'un module (code + tests + doc)

En tant que **tech lead**, je veux **demander une analyse complète d'un module (code + tests + doc)**
afin de *avoir une vue 360° de la qualité en une commande*

Critères d'acceptation :

- ✓ POST /orchestrate avec { task, target }
- ✓ Orchestrateur appelle : CodeAnalyzer → TestRunner → DocWriter
- ✓ Chaque agent reçoit le contexte et les résultats des précédents
- ✓ Réponse agrège les rapports avec summary

M6-US02 spécifier quels agents utiliser

En tant que **développeur**, je veux **spécifier quels agents utiliser**
afin de *contrôler la granularité de l'orchestration*

Critères d'acceptation :

- ✓ agents[] permet de choisir un sous-ensemble
- ✓ Si omis → 3 agents par défaut
- ✓ Agent inconnu → 400 avec liste valides

M6-US03 que l'orchestration échoue gracieusement

En tant que **développeur**, je veux **que l'orchestration échoue gracieusement**
afin de *obtenir les résultats partiels*

Critères d'acceptation :

- ✓ Agent en échec → { status: 'error', error: string }
- ✓ Agents suivants continuent
- ✓ Summary mentionne les échecs
- ✓ duration_ms mesure le temps total

User Stories — Frontend Orchestration

M6-US04 voir la progression de l'orchestration en temps réel dans le chat

En tant que **développeur**, je veux **voir la progression de l'orchestration en temps réel dans le chat**
afin de *suivre quels agents travaillent et lesquels ont terminé*

FRONTEND *React ou Angular — au choix du participant*

Critères d'acceptation :

- ✓ Un composant OrchestratorProgress s'affiche sous le message de lancement
- ✓ Il montre une timeline verticale avec les 3 agents : ⏳ En cours / ✓ Terminé / ❌ Erreur
- ✓ L'agent actif est mis en surbrillance
- ✓ Le temps écoulé est affiché en temps réel

M6-US05 voir les rapports de chaque agent dans des cartes dédiées

En tant que **développeur**, je veux **voir les rapports de chaque agent dans des cartes dédiées**
afin de *consulter les résultats détaillés de l'analyse, des tests et de la doc*

FRONTEND *React ou Angular — au choix du participant*

Critères d'acceptation :

- ✓ Chaque agent terminé produit une carte AgentReport dans le chat
- ✓ La carte affiche : nom de l'agent, statut, durée, résumé des findings
- ✓ Les cartes sont collapsibles pour ne pas surcharger le chat
- ✓ Le rapport complet est accessible en dépliant la carte
- ✓ Un summary global est affiché en conclusion par Assist

5. Exigences non fonctionnelles

5.1 Performance

- Temps de réponse /health : < 50ms
- Temps de réponse /search : < 500ms pour 100 fichiers
- Premier token streaming : < 2s après envoi
- Time to Interactive du frontend : < 3s (Lighthouse)
- Rendu streaming fluide : pas de lag perceptible entre les tokens
- Orchestration multi-agents : < 60s pour 3 agents séquentiels

5.2 Sécurité

- Clé API en variable d'environnement, jamais dans le code ni le frontend
- Le frontend n'appelle jamais l'API Anthropic directement (tout passe par le backend)
- Path traversal bloqué sur tous les endpoints fichiers
- Entrées validées (longueur, format, caractères spéciaux)
- run_tests n'exécute que la commande configurée (pas d'injection)
- CORS configuré pour autoriser uniquement le frontend local

5.3 UX / Accessibilité

- Le chat est responsive (fonctionne sur desktop et tablette, min-width: 768px)
- Les contrastes respectent WCAG AA (ratio minimum 4.5:1)
- Le focus clavier fonctionne dans le champ de saisie et les boutons
- Les états de chargement sont toujours visibles (skeleton, spinner, curseur)
- Le dark mode est supporté (optionnel — bonus avancé)

5.4 Maintenabilité

- Backend : Clean Architecture (routes/, services/, schemas/, config/)
- Frontend React : composants dans src/components/, hooks dans src/hooks/
- Frontend Angular : composants dans src/app/components/, services dans src/app/services/
- Tests frontend : au minimum 1 test par composant critique (ChatContainer, MessageInput, StreamingText)

- README.md à jour avec instructions de setup backend ET frontend

5.5 Portabilité

- Backend : macOS, Linux, Windows — 4 langages supportés
- Frontend : Chrome, Firefox, Safari, Edge (dernières 2 versions)
- Aucune dépendance système externe (pas de Docker, pas de BDD)

6. Contrat d'API — Récapitulatif

Méthode	Endpoint	Module	Description
GET	/health	M1	État du serveur
GET	/config	M1	Configuration courante
POST	/config	M1	Modifier la configuration
POST	/chat	M2	Message (bloquant)
POST	/chat/stream	M2	Message (streaming SSE)
GET	/chat/history	M2	Historique conversation
DELETE	/chat/history	M2	Réinitialiser historique
POST	/search	M3	Recherche documentation
POST	/chat (+ tools)	M4	Chat avec tool calling
POST	/analyze-screenshot	M4	Analyse d'image
POST	/orchestrate	M6	Tâche multi-agents

7. Roadmap de développement

L'ordre d'implémentation suit la progression de la formation. Le frontend est intégré progressivement.

Phase	Backend	Frontend	Session
Phase 1 — Fondations	M1 : /health, /config	Setup projet React/Angular	S1-S2
Phase 2 — Search	M3 : POST /search	—	S3
Phase 3 — Qualité	Refactoring Clean Arch	—	S4
Phase 4 — Chat	M2 : /chat + /chat/stream	ChatContainer, MessageBubble, StreamingText, MessageInput	S5
Phase 5 — Tools	M4 : 3 tools + chaînage	ToolIndicator, ToolResultCard, ImageUpload	S6
Phase 6 — MCP	M5 : Serveur MCP	—	S7
Phase 7 — Agents	M6 : 3 agents + orchestrateur	AgentReport, OrchestratorProgress	S8

Stratégie frontend progressive

Phase 1-3 (Jour 1) : Le backend est développé et testé via curl/Postman. Le projet frontend est créé (scaffolding) mais pas connecté.

Phase 4 (Jour 2 matin) : Le frontend chat est connecté au backend. Le streaming fonctionne end-to-end.

Phase 5 (Jour 2 après-midi) : Les composants tools et l'upload d'image sont ajoutés au chat.

Phase 7 (Jour 2 fin) : L'affichage de l'orchestration multi-agents est ajouté.

8. Glossaire

Terme	Définition
SPA	Single Page Application — application web monolithique chargée une seule fois
SSE	Server-Sent Events — protocole de streaming serveur → client
Claude API	API REST d'Anthropic pour interagir avec les modèles Claude
Function Calling	Capacité de Claude à appeler des fonctions (tools) définies par le développeur
Tool	Fonction exécutable par Claude via le function calling
MCP	Model Context Protocol — standard d'interopérabilité pour les outils IA
System Prompt	Instructions invisibles guidant le comportement de Claude
Token	Unité de texte (~4 caractères) pour mesurer les entrées/sorties LLM
Streaming	Envoi de la réponse token par token plutôt qu'en bloc
Agent	Instance autonome de Claude avec un rôle et des tools dédiés
Orchestrateur	Composant coordonnant l'exécution de plusieurs agents
Vision API	Capacité de Claude à analyser des images
EventSource	API JavaScript native pour consommer des flux SSE
Tailwind CSS	Framework CSS utility-first pour un styling rapide
Angular Material	Bibliothèque de composants UI pour Angular (Material Design)

Note pour la décomposition en stories

Les stories FRONTEND sont identifiées par le tag violet dans le document.

Total : 27 User Stories (19 backend + 8 frontend) décomposables en ~50-60 sous-tâches.

Les critères d'acceptation servent de base directe pour les tests.

L'ordre suit la roadmap (section 7) : backend d'abord, frontend intégré progressivement.

Chaque story frontend dépend de la story backend correspondante du même module.